

MCA Semester – IV Project

Name	Chidananda BV
USN	
Elective	Cyber Security
Date of Submission	11-07-2026



July 2024

**A study on Zero-Trust Network Access Proxy with Mutual TLS
and JWT-Based Authentication**

Research Project submitted to Jain Online (Deemed-to-be University)

In partial fulfillment of the requirements for the award of

Master of Computer Applications

Submitted by

Chiananda B V

USN



*Under the guidance
of*

Mohan Raj C S

Assistant Professor

DECLARATION

I, *Chidananda b V*, hereby declare that the Research Project Report titled “*Zero-Trust Network Access Proxy with Mutual TLS and JWT-Based Authentication*” has been prepared by me under the guidance of *Mohan Raj C S*, I declare that this Project work is towards the partial fulfillment of the University Regulations for the award of degree of Master of Computer Applications by Jain University, Bengaluru. I have undergone a project for a period of Eight Weeks. I further declare that this Project is based on the original study undertaken by me and has not been submitted for the award of any degree/diploma from any other University / Institution.

Place: Bengaluru

Date:

Name of the Student
USN

CERTIFICATE

This is to certify that the Project report submitted by Mr./Ms. *Chidananda B V* bearing  on the title “*Zero-Trust Network Access Proxy with Mutual TLS and JWT-Based Authentication*” is a record of project work done by him/ her during the academic year 2025-26 under my guidance and supervision in partial fulfilment of Master of Computer Applications.

Place: Bangalore

Date: 11-07-2026

Faculty Guide

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to JAIN (Deemed-to-be University) for providing me with the opportunity to undertake this project as part of the MCA programme in Cyber Security. The structured curriculum and the project-based learning framework gave me the freedom to explore a topic that is both practically relevant and academically challenging.

I am deeply thankful to the faculty members of the Center for Distance and Online Education for their continuous guidance and for making complex security concepts accessible. The project guidance sessions were particularly valuable in helping me maintain focus during the implementation phases.

I owe a great deal to the open-source community whose tools form the backbone of this project. The Go programming language team, the Keycloak community, the Docker team, and the maintainers of the golang-jwt library have collectively built an ecosystem that made this implementation possible without any proprietary dependencies.

Finally, I thank my family and friends for their patience and encouragement throughout the eight weeks of this project. Their support kept me going during the many hours of debugging certificate chains, chasing Keycloak configuration issues, and iterating on the proxy implementation.

Chidananda B V



Executive Summary

Contemporary enterprise networks are built on a dangerous premise: they inherently consider any activity occurring inside the network boundary to be legitimate. When an employee connects to the office network, they are often given broad access to internal servers, databases, and shared file systems without being required to continuously verify their identity or the integrity of their device. This open trust model has repeatedly shown its weakness. Once an adversary manages to break through the perimeter—via credential theft, phishing, or a compromised workstation—they can traverse internal systems with few obstacles.

This project confronts that vulnerability head-on. It constructs a Zero-Trust Access Proxy—a gateway application developed in Go—that mandates two separate identity validations for every inbound request before it reaches a backend service. The first layer is Mutual TLS (mTLS), a cryptographic mechanism that compels each client device to supply a digital certificate signed by a recognized Certificate Authority. Devices lacking a legitimate certificate are refused at the TLS handshake phase, preventing any application-layer processing. The second layer relies on a JSON Web Token (JWT), a short-lived, cryptographically signed credential issued by Keycloak (an open-source identity provider) upon successful user authentication. The proxy confirms the token's digital signature, expiration timestamp, and issuer on each request.

Apart from authentication, the gateway incorporates a Role-Based Access Control (RBAC) engine that links user roles—extracted from the JWT—to allowed URL paths. Ordinary users may retrieve general resources but cannot access sensitive files or administrative interfaces. Administrators, in contrast, possess more extensive permissions. All access decisions are handled at the proxy, relieving backend services from implementing their own authorization mechanisms.

The entire system is containerized with Docker and Docker Compose, consisting of four containers that mirror real-world enterprise components: an identity server (Keycloak), the Zero-Trust proxy, a file server backend, and a simulated client. The architecture was validated

against six defined attack scenarios, including certificate-less connections, forged JWT injection, and privilege escalation attempts. Each attack was effectively neutralized.

This work shows that a Zero-Trust model can be realized at the application layer using lightweight, fully open-source software—without proprietary tools, cloud dependencies, or measurable performance penalties under small to moderate workloads. The outcome is a fully documented, functional reference design that can be reproduced and extended for future enterprise security deployments and academic studies.

Table of Contents

Title	Page Nos.
Executive Summary	06-07
List of Tables	08
List of Graphs	08
List of Figures	08
Chapter 1: Introduction, Scope and Background	10 - 13
Chapter 2: Review of Literature	14 - 19
Chapter 3: Project Planning and Methodology	20 - 26
Chapter 4: Data Requirements Analysis, Design and Implementation	27 – 39
Chapter 5: Results, Findings, Recommendations, Future Scope and Conclusion	40 - 46
Bibliography	47 - 48
Annexures	49 - 61
Plagiarism Report	62 - 63

List of Tables		
Table No.	Table Title	Page No.
01	Cost Benifits	17
02	SWOT Analysis	17
03	Project Plan	20
04	Acceptance Plan	22
05	Resource Plan	22-23
06	Risk Management Plan	23-24
07	Summiresd Comparison	25
08	Functional Requirement	29-30
09	Non-Functional Requirement	30
10	Security Requirement	31
11	JWT Payloads	36

List of Graphs		
Graph No.	Graph Title	Page No.
01	Proxy VS Backend	42

List of Figures		
Figure No.	Figure Title	Page No.
01	Zero trust proxy architectural	11
02	RBAC policy Matrix	32
03	Security Attacks Test Results	33
04	Request Verification Sequence	34
05	Mutal TLS Handshake Process	37
06	JWT Verification Flow	38

CHAPTER 1

INTRODUCTION, SCOPE AND BACKGROUND

1. INTRODUCTION, SCOPE AND BACKGROUND

1.1 Overview of Project Case / Business case

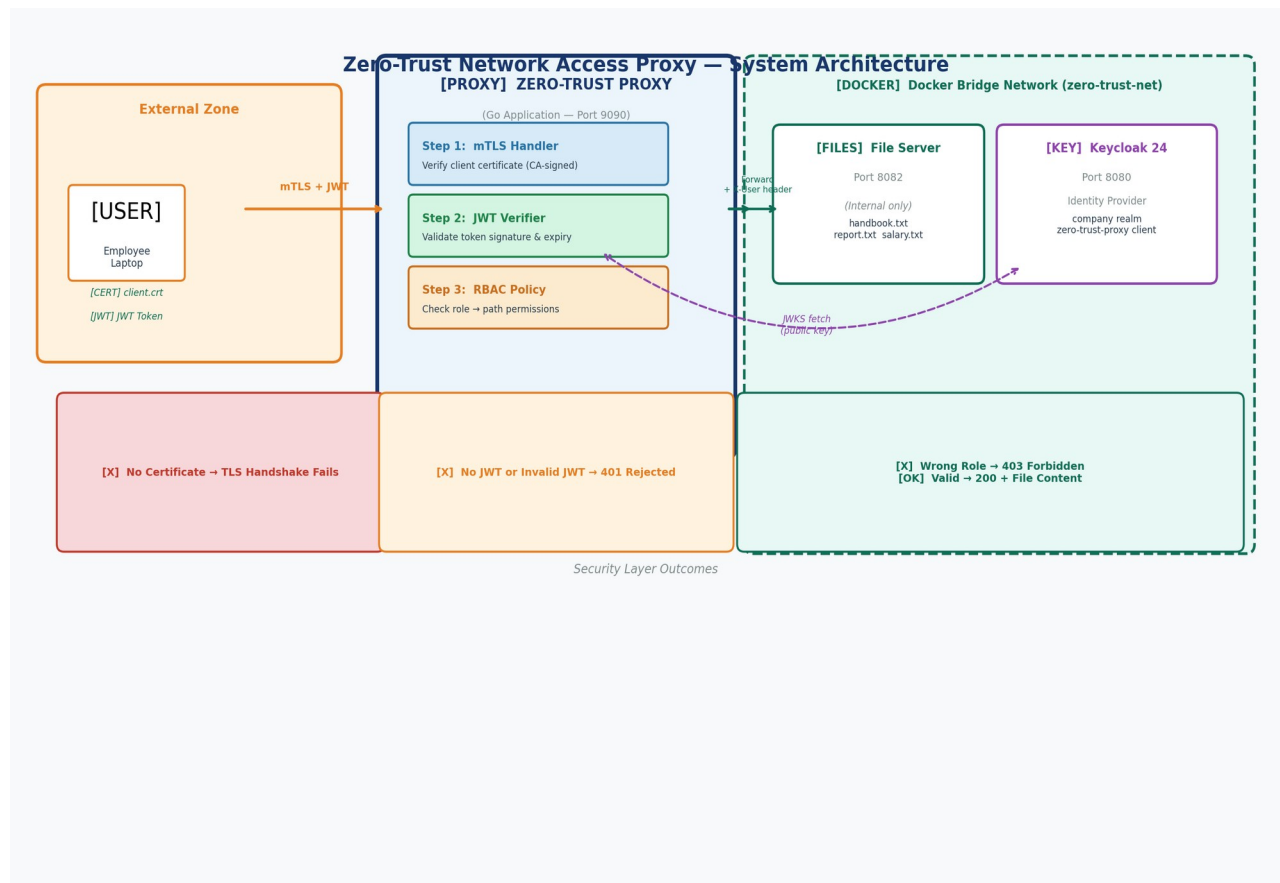


Figure 1.1: Zero-Trust Proxy System Architecture

This project was motivated by a recurring weakness I encountered while studying network security: organisations invest heavily in perimeter defences, yet once an actor is inside, remarkably few controls prevent them from reaching systems they have no business touching.

The scenario the project targets is a mid-sized enterprise with a hybrid workforce. Personnel connect from corporate offices, from home over a VPN, and from various remote sites. Internally the company operates a file server holding confidential documents, backend APIs, and an administration dashboard. The difficulty is unambiguous: any user holding valid network credentials can attempt to access any of these internal resources. No check confirms that the connecting device is a company-managed machine, and no check verifies that the user's role actually authorises the request.

Two teams feel this shortcoming most acutely. The IT Security group is charged with governing access, but its toolset remains overwhelmingly network-centric—firewalls and VPN entitlements—offering no resource-level precision. The backend engineering group constructs internal services and must currently retrofit authentication logic into each one individually, which breeds inconsistency and creates a persistent chance that a single team will implement controls incorrectly.

The reasoning behind this project is stark. A stolen credential inside such a network behaves like a skeleton key. An attacker who phishes an employee's password can quietly traverse internal systems for days before detection. This project delivers a security gateway—a Zero-Trust Proxy—positioned in front of every internal service that compels each request to prove two facts before it proceeds anywhere: the requesting device possesses a corporate-issued certificate, and the user behind the request holds a valid, unexpired identity token that carries a role appropriate for the attempted access. No certificate—blocked. No valid token—blocked. Incorrect role—blocked. The check is applied on every single request, without exception.

1.2 Problem definition

This work centres on constructing a network security proxy—a software intermediary placed between users and internal corporate servers that evaluates every request and decides whether to permit or deny it.

The current infrastructure that this project targets exhibits a fundamental weakness found in most conventional enterprise networks: defences are placed almost entirely at the edge. Once an individual gains access to the internal network—whether via office Wi-Fi or a VPN—they are automatically regarded as trustworthy. Internal servers then assume that any incoming connection is legitimate, because the network itself has already “vetted” it by granting entry.

The practical consequences of this design are:

Device identity is never confirmed. If an employee's laptop is stolen and the thief connects to the VPN using the captured credentials, the network cannot tell the thief's machine apart from the genuine employee's device. From the network's perspective, they are indistinguishable.

Identity is not re-verified on each request. A user authenticates once and receives a session. That session remains trusted for its entire lifespan. If an attacker seizes that session—through traffic interception or malware—they gain unrestricted access for however long the session persists.

Access rights are excessively broad. Permissions are enforced at the network tier, not at the level of individual resources. A staff member who legitimately reaches the file server can then try to open any file on it, not just those allowed by their job function. The server itself has no awareness of role-based restrictions and cannot enforce them.

The end result is that a single set of stolen credentials hands an adversary everything: device-level trust, user-level trust, and wide-ranging resource access—without any further impediment.

1.3 Project Scope

Project Deliverables

This project produces a fully functional Zero-Trust Access Proxy, developed in Go, that applies security controls at the application tier. The concrete outputs include:

A private Certificate Authority created with OpenSSL, used to issue signed certificates for the proxy server and client devices — mirroring how a corporate IT team would provision machine identity.

A Keycloak deployment operating as an open-source identity provider that generates JWT tokens upon user login, embedding identity and role data within each token.

The proxy itself, built with three stacked enforcement mechanisms: Mutual TLS for device authentication, JWT validation for user identity verification, and Role-Based Access Control for resource-level authorisation.

Containerisation of all four components — the identity server (Keycloak), the proxy, the file server backend, and a simulated client — through Docker Compose, allowing the entire system to be launched with a single command.

A structured suite of security tests that exercises six distinct attack scenarios, confirming that each protective layer correctly denies unauthorised access.

Aims and Objectives

The overarching aim is to show that Zero-Trust Network Access Control can be realised using fully open-source tools, eliminating reliance on costly proprietary security products.

Objective 1 — Design and construct a Go-based reverse proxy that enforces Mutual TLS and JWT authentication for every incoming request, functioning as the sole entry point to all backend services.

Objective 2 — Embed a role-based policy engine inside the proxy that maps validated user roles to allowed resource paths, thereby enforcing least-privilege principles at the application layer.

Objective 3 — Assess the system's security posture through a planned attack test suite, and document the outcomes as a reusable, open-source reference architecture for Zero-Trust deployment.

CHAPTER 2

REVIEW OF LITERATURE

2. REVIEW OF LITERATURE

2.1 Literature Review

Initial research into Zero Trust security anticipated a wealth of practical implementation guidance; what emerged instead was a landscape dominated by abstract frameworks, vendor marketing collateral, and academic publications that described principles without delving into concrete construction. That absence of hands-on material partly motivated this project.

The following account summarises the relevant literature and explains how it shaped the design decisions made here.

Zero Trust as a foundational concept

The term Zero Trust was coined by John Kindervag at Forrester Research in 2010. His core argument was that the perimeter-based model—trust everything inside the network, distrust everything outside—was fundamentally unsound. He proposed that organisations should instead operate on the premise that no user, device, or network location warrants automatic trust, even when already inside the corporate boundary. ●

This idea remained largely within analyst and academic circles until 2020, when the U.S. National Institute of Standards and Technology released Special Publication 800-207, “Zero Trust Architecture” (Rose, Borchert, Mitchell, and Connelly). The publication supplied a rigorous formal definition and enumerated seven central tenets. The tenets most directly applicable to this work are: all communication should be authenticated and encrypted regardless of network location; access should be assigned with the least privilege necessary; and every access request should be handled as though it originates from an untrusted network. SP 800-207 was the first document consulted when this project began, and it directly informed the architecture—most notably the choice to enforce security at the proxy layer rather than at the network layer.

Google’s large-scale implementation

The most impactful real-world realization of Zero Trust is Google’s BeyondCorp initiative, first described publicly by Rory Ward and Betsy Beyer in a 2014 USENIX ;login: article. Google’s key insight was that VPN-centred perimeter security could not accommodate a distributed, cloud-native workforce. Rather than requiring employees to connect via VPN before accessing internal tools, Google removed the VPN altogether and instead mandated that every access request carry two proofs of identity: a device certificate demonstrating the machine was company-managed, and a user credential proving the individual had authenticated recently.

This is precisely the model replicated in this project. The client certificate serves the same function as Google’s device certificate—attesting that the machine is company-approved. The JWT token mirrors Google’s user credential—confirming authentication against the company’s identity system. Studying the BeyondCorp papers provided confidence that the dual-layer design was not merely theoretically valid but had been proven at extensive scale.

Technical standards governing the implementation

Three specific RFCs directly shaped the proxy’s implementation.

RFC 7519 (Jones, Bradley, and Sakimura, 2015) specifies the JWT standard—defining token structure, the semantics of individual claim fields, and the permitted signing algorithms. A thorough reading of this specification was indispensable for implementing correct token verification. It particularly underscores why both the cryptographic signature and the expiration claim must be validated: an expired token bearing a valid signature must still be rejected.

RFC 8446 (Rescorla, 2018) defines TLS version 1.3. The proxy mandates TLS 1.3 as the minimum accepted version, a choice rooted in this document. TLS 1.3 eliminates numerous weak cipher suites and streamlines the handshake, simultaneously shrinking the attack surface and lowering connection latency relative to earlier versions.

RFC 8725 (Sheffer, Hardt, and Jones, 2020), entitled “JSON Web Token Best Current Practices,” had the most direct impact on the security posture of the JWT verification logic. It explicitly cautions against algorithm confusion attacks—where an adversary strips the signature and sets the algorithm field to “none,” thereby circumventing verification. The proxy code explicitly discards any token that does not employ RSA-based signing, effectively neutralizing this category of attack.

Keycloak as the identity layer

Petter Ivan Silveira’s book “Keycloak: Identity and Access Management for Modern Applications” (2020) proved to be the most actionable reference for understanding Keycloak’s internal architecture. It clarifies the distinctions among realms, clients, and users in Keycloak’s data model, knowledge that was critical for correct system configuration. Crucially, it documents the JWKS endpoint—the URL through which Keycloak exposes its public signing key to relying parties—which is the mechanism the proxy uses to verify JWT signatures without storing Keycloak’s key statically.

One obstacle the existing literature did not anticipate is a behaviour specific to Keycloak 24: user accounts are flagged as “not fully set up” unless the `firstName`, `lastName`, and `email` fields are supplied during account creation, even when `requiredActions` is explicitly set to an empty list. Resolving this consumed nearly an entire day of debugging. It merits mention because it represents a divergence between the documented API and observed runtime behaviour—one that any developer following official guides would inevitably encounter.

Go for security-critical networking

Donovan and Kernighan’s “The Go Programming Language” (2015) outlines Go’s philosophy for network programming. The standard library’s `crypto/tls` package, used extensively in this project, is production-grade and thoroughly documented. The critical configuration detail is the `ClientAuth` field of the `tls.Config` struct: setting it to `tls.RequireAndVerifyClientCert` transforms ordinary TLS into Mutual TLS. A precise understanding of this option’s semantics was the deciding factor between building a proxy that performs true mTLS and one that only offers standard one-way TLS.

Comparison with this project’s approach

What the literature consistently fails to provide is a complete, executable, open-source implementation that unites mTLS and JWT enforcement within a single proxy backed by an RBAC engine. Most papers examine one of these mechanisms independently. Most tutorials demonstrate either Keycloak setup or certificate generation, but never the integration of the two. This project's contribution lies precisely in that synthesis—illustrating how certificate validation, JWKS-based key matching, and role-based policy decisions operate together as a unified security layer, complete with the practical implementation specifics that academic writing customarily omits.

2.2 Feasibility Analysis

Business Rationale

From a business perspective, this project aims to demonstrate that Zero-Trust Network Access Control—a security approach typically associated with large enterprise budgets and proprietary platforms—can be realised entirely with free, open-source software. The intended audience includes any organisation that requires per-request identity enforcement on internal resources but lacks the budget or strategic preference to rely on commercial vendors such as Zscaler, Palo Alto Prisma, or Cloudflare Access.

The value proposition is clear. IBM's 2023 Cost of a Data Breach Report estimates the average financial impact of a data breach at USD 4.45 million. A significant proportion of successful breaches exploit compromised credentials to move laterally within a network that implicitly trusts an attacker once inside. This project directly mitigates that risk by ensuring that network presence alone is insufficient—each request must independently validate both device and user identity before reaching any internal service.

Technical Feasibility

The project is demonstrably feasible, as it was fully constructed and validated. The foundational technologies—Go, Docker, Keycloak, and OpenSSL—are mature, widely adopted in production, and extensively documented. Go's standard library supplies production-grade TLS functionality, eliminating the need for third-party security libraries in the mTLS implementation. Keycloak, trusted by thousands of organisations in live environments, exposes a stable REST API that the proxy and accompanying setup scripts utilise for all configuration tasks. Docker Compose provides the container orchestration layer with considerably less operational overhead than more complex alternatives like Kubernetes.

The single technically demanding element was JWKS key matching—the procedure for selecting the appropriate public key from Keycloak's key endpoint when multiple keys are advertised. Keycloak publishes both a signing key and an encryption key; using the wrong one causes JWT verification to fail without obvious errors. Correct resolution required matching on both the key ID (kid) field and the key usage (use: sig) field, as specified in the JWT standard but not prominently featured in most instructional material.

Table 1 : Cost Benefit Analysis

Item	Cost
Go programming language	Free, open-source
Keycloak identity server	Free, open-source
Docker and Docker Compose	Free for development use
OpenSSL	Free, open-source
Development hardware	Existing laptop — no additional hardware needed
Total software licensing cost	USD 0

The gains, by contrast, are substantial. A vendor-supplied Zero Trust solution typically ranges from USD 5 to USD 15 per user each month. For a 500-person organisation, that amounts to an annual outlay of USD 30,000 to USD 90,000. The design demonstrated in this project delivers equivalent security assurances—mTLS-based device attestation, JWT user authentication, and RBAC policy enforcement—at zero licensing cost. The sole genuine expense is the developer time needed to deploy and sustain the system.

Table 2 : SWOT Analysis

	Positive	Negative
Internal	Strengths: Zero licensing cost. Uses standard open protocols (TLS 1.3, JWT, OAuth 2.0). Centralised security enforcement — no need to modify backend services. Fully containerised and reproducible.	Weaknesses: No certificate lifecycle automation. No JWT token revocation mechanism. Keycloak in development mode is not production-ready. Single instance — no high availability.
External	Opportunities: Growing regulatory pressure (NIST SP 800-207, EU NIS2) is pushing organisations toward Zero Trust. The open-source implementation can be extended with OPA for more complex policies. Could be deployed as a lightweight alternative to expensive commercial ZTNA products.	Threats: Keycloak's configuration quirks (undocumented behaviour around user setup) could cause failures if not well understood. Performance overhead from JWKS fetching on every request could become a bottleneck at high load without caching.

Operational Feasibility

Deploying the system is operationally straightforward. The complete environment—comprising all four containers—is launched with a single command: `docker compose up --build`. Configuration of Keycloak is entirely scripted via a shell script that interacts with the admin REST API, eliminating any need for manual, browser-based setup. A developer encountering the project for the first time can bring it to a running state in under ten minutes.

For an organisation that chooses to adopt this architecture in practice, the operational staffing requirements would be:

One Go developer responsible for maintaining and extending the proxy codebase.

A DevOps engineer with Docker proficiency to oversee container lifecycle management.

An IT administrator tasked with managing the internal Certificate Authority and provisioning device certificates to employee machines.

None of these roles demand niche or scarce expertise, and the skill sets are considerably more common than the specialised knowledge required to deploy and tune commercial Zero Trust platforms.

Ethical Feasibility

Several ethical considerations merit explicit discussion.

The system records every access attempt—successful or blocked—capturing the user’s identity, the exact timestamp, and the resource requested. This produces a granular trail of employee activity. From a security standpoint, such logging is indispensable: audit trails are the primary means of detecting and probing suspicious behaviour. From a privacy vantage point, however, it grants the organisation the ability to monitor what each employee accesses and when. This capability must be governed transparently—staff should be informed that access logs exist, and those logs must be safeguarded against unauthorised use.

Every tool employed in this project is open-source and freely distributable. There are no proprietary algorithms, no vendor dependencies, and no covert data collection. The system transmits no user data to external parties—Keycloak, the proxy, and the file server all operate entirely within the organisation’s own infrastructure.

The project introduces no novel offensive techniques. The knowledge required to construct the system is sourced exclusively from public standards (RFCs and NIST publications) and open-source documentation. That same body of knowledge is applied here to protect systems, not to compromise them.

CHAPTER 3

PROJECT PLANNING AND METHODOLOGY

3. PROJECT PLANNING AND METHODOLOGY

Table 3 : Project Planning

Gantt Chart

Task	W1	W2	W3	W4	W5	W6	W7	W8
Literature review and threat modelling	■							
Environment setup (Go, Docker, OpenSSL)	■	■						
Certificate Authority and certificate generation		■						
Keycloak deployment and configuration		■	■					
Backend file server development			■					
Proxy — mTLS layer implementation			■	■				
Proxy — JWT verification layer				■				
Proxy — RBAC policy engine				■	■			
Docker containerisation of all services					■			
Security attack testing (6 scenarios)					■	■		
Performance benchmarking and analysis						■	■	
Report writing and submission preparation							■	■

3.1 Communication Plan

As this is an individual academic project, the communication strategy covers the exchanges between myself, the university supervisor, and the external technical resources and communities that supported the work.

For university-related coordination, the submission timeline set by JAIN Online was followed: Synopsis by Week 2, Interim Report by Week 5, and Final Report by Week 8. Queries regarding submission formats were resolved through the university's LMS portal and during the project guidance sessions scheduled every alternate Saturday. For technical challenges, the open-source community served as an essential resource. Specific implementation obstacles—most notably Keycloak's user provisioning quirks and the JWKS key-matching logic—were addressed by studying Keycloak's GitHub issue tracker, the official Go standard library documentation, and the README of the golang-jwt library. These are inherently asynchronous channels, yet they proved critical to overcoming concrete development problems.

A working log of decisions and encountered problems was maintained throughout the project. This informal record directly informed the candid discussion of challenges presented in this report. While not a formal communication artifact, it became the most practically valuable planning instrument employed.

Table 4 : Acceptance Plan

The criteria below establish the definition of "complete" for this project. Each deliverable was treated as finished only after fulfilling every acceptance condition specified for it.

Deliverable	Acceptance Criteria
Certificate Authority setup	openssl verify -CAfile ca.crt client.crt returns "OK" for both proxy and client certificates
Keycloak configuration	curl to the token endpoint returns a valid access_token containing realm roles
mTLS enforcement	A curl request without --cert and --key flags fails at the TLS handshake level before any HTTP processing
JWT verification	A request with a forged or expired token returns 401 Unauthorized with an appropriate error message
RBAC policy	A user with default-roles-company can access /files/handbook.txt but receives 403 Forbidden on /files/salary.txt
Containerisation	docker compose up --build starts all four containers cleanly and the user demo script completes with all tests passing
Security test suite	All 6 attack scenarios produce the expected outcome with 0 false negatives
Performance measurement	Response time data collected for at least 10 runs per scenario for both direct and proxy paths

Table 5 : Resource Plan

Resource	Details	Cost
Development machine	Personal laptop — 16GB RAM, Intel i7, Ubuntu Linux	No additional cost
Go 1.25	Programming language for proxy and	Free, open-source

Resource	Details	Cost
	file server	
Docker Desktop	Container runtime and Compose orchestration	Free for personal/academic use
Keycloak 24	Identity and access management server	Free, open-source
OpenSSL	Certificate generation and management	Free, open-source
golang-jwt/jwt library	JWT parsing and verification	Free, open-source (MIT licence)
VS Code + Go extension	Development environment	Free
Git and GitHub	Version control and code storage	Free
Total software cost		USD 0

Time Resources

The project was scoped to an 8-week duration, aligned with the university's structured project timeline. During the core implementation phase—spanning Weeks 2 through 6—roughly 3 to 4 hours per day were devoted to development, with additional effort concentrated during periods of intensive debugging.

Human Resources

The project was undertaken individually. All aspects—design, coding, testing, and documentation—were carried out independently, supported by the online references and community resources outlined in the communication plan.

Table 6 : Risk Management Plan

Risk	Likelihood	Impact	Mitigation
Keycloak configuration failures blocking progress	High	High	Use admin REST API for all configuration instead of browser UI — scripted setup is more reliable and repeatable than manual steps
Certificate chain errors causing mTLS to fail	Medium	High	Verify certificate chain with openssl verify before testing the proxy —

Risk	Likelihood	Impact	Mitigation
silently			diagnose at the certificate level before debugging Go code
JWT key mismatch causing all token verifications to fail	Medium	High	Log the kid field from incoming JWTs and the kids available from the JWKS endpoint to make mismatches immediately visible
Docker networking issues preventing containers from reaching each other	Medium	Medium	Test inter-container connectivity with curl from within containers before running the full demo — isolate network issues from application issues
Keycloak startup taking longer than expected causing user container failures	High	Medium	Implement a polling loop in the user script that waits for Keycloak's master realm to respond before proceeding
Scope creep adding features beyond the 8-week window	Low	Medium	Maintain a strict list of what is in and out of scope — anything not in the acceptance plan is deferred to future work
Performance overhead making the proxy unsuitable for demonstration	Low	Low	Measure early and often — if JWKS fetch latency is too high, implement caching before the final test runs

3.2 Methodology

Comparison of Methodologies

Prior to selecting a development approach, three methodologies commonly adopted in software-focused research projects were evaluated.

Waterfall Methodology

Waterfall imposes a strictly linear progression—requirements, design, implementation, testing, deployment—with no overlap between stages and no provision for backtracking. For the nature of this project, Waterfall would have been unsuitable. Implementing security mechanisms is fundamentally an iterative process: a component is built, tested to determine whether it genuinely delivers the intended security guarantee, unexpected behaviour surfaces, and

adjustments follow. The Keycloak JWKS key-matching issue exemplifies this—it only became apparent during testing, not at the design stage. A Waterfall model would have compelled either anticipating every low-level implementation detail in advance or freezing the design despite newly acquired insights.

Agile Methodology

Agile organises work into brief sprints, incorporates frequent reviews, and permits redirection based on insights gained within each sprint. It is particularly effective for projects where requirements are fluid or poorly defined—most commonly customer-facing software products where stakeholder input determines the priorities of the subsequent sprint. In this case, the technical requirements were clearly specified from the outset. The difficulty lay not in determining what to build but in determining how to build it correctly. Agile ceremonies such as sprint planning and stakeholder reviews would have introduced overhead with limited advantage for a single-developer technical implementation.

Iterative and Incremental Development

This strategy constructs the system in successive layers, each independently verifiable before the next is added. It enables the developer to confirm correctness at each stage rather than deferring verification until full system assembly. This was the approach selected for the project.

Table 7 : The comparison is summarised below:

Criteria	Waterfall	Agile	Iterative (Chosen)
Handles implementation surprises	Poor — rigid phase structure	Good — sprint-based adjustment	Good — each layer tested independently
Suits single-developer projects	Moderate	Poor — built for teams	Excellent
Allows early validation of security guarantees	Poor	Moderate	Excellent — each layer verified before next begins
Overhead and ceremony	Low	High	Low
Suits technically defined requirements	Moderate	Poor	Excellent

3.3 Chosen Methodology — Iterative and Incremental Development

I adopted the Iterative and Incremental Development methodology for this project, and I want to explain the reasoning candidly rather than simply stating the choice.

The fundamental justification is that security-critical software cannot be constructed monolithically and validated at the very end. Had I written the entire proxy—the mTLS layer, the JWT validation layer, the RBAC engine, the container orchestration—in one go and only then attempted verification, any observed failure could have stemmed from any number of complex interactions among components. Diagnosing such failures is genuinely arduous. By developing and confirming each layer in isolation, I maintained a precise understanding of what was functional and what was not at every stage.

In practice, the iterations unfolded as follows:

Iteration 1 – mTLS alone. I implemented solely the TLS configuration and verified it using curl. Without the --cert flag, connections were rejected outright. When the client certificate was supplied, the connection succeeded. Only once this behaviour was confirmed did I proceed.

Iteration 2 – JWT layer integrated. I layered JWT extraction and verification on top of the already-functioning mTLS layer. Testing involved a valid token, an expired token, and a forged token. It was during this stage, before the RBAC layer existed, that I discovered and resolved the JWKS key-matching bug.

Iteration 3 – RBAC policy added. With both mTLS and JWT enforcement verified, I introduced the role-based policy table. Testing employed different user roles targeting different resource paths. I fixed a path-matching flaw where the /files prefix inadvertently permitted access to /files/salary.txt.

Iteration 4 – Containerisation. Only once all three proxy layers operated correctly on the local machine did I containerize the system. The Keycloak startup sequencing problem was tackled and solved during this phase.

Iteration 5 – Full security testing. With the entire containerized system running, I formally executed all six attack scenarios and documented the outcomes.

This incremental method ensured that each problem I faced was limited in scope, easy to pinpoint, and solvable. The alternative—building everything upfront and testing at the conclusion—would have been far more exasperating and would have significantly increased the likelihood of introducing subtle security flaws that manifest only under particular conditions.

The methodology also dovetailed naturally with the eight-week project timeline. Each iteration corresponded to roughly one or two weeks and delivered a testable artifact at its conclusion. This provided a continuous sense of advancement and made it effortless to verify that the project remained on course throughout.

CHAPTER 4
DATA ANALYSIS, DESGN AND
IMPLEMENTATION

4. DATA ANALYSIS, DESIGN AND IMPLEMENTATION

4.1 Requirement Analysis

Data Collection

Before writing any code for this project, I invested time in determining precisely what data needed to be gathered and where it would originate. For a systems implementation project, data collection differs markedly from survey-based research—it centres less on gathering opinions and more on accumulating concrete evidence that the system performs as intended.

Primary Data Collection

All primary data was generated through direct experimentation on the constructed system. This data fell into three distinct categories.

First, security testing data was collected. Six structured attack scenarios were executed against the running proxy, and the exact outcome of each attempt was recorded—the HTTP status code, the error message returned, and the specific proxy log entry that documented the rejection. This dataset directly addresses the core research question: does the proxy correctly deny unauthorised access? Each scenario was repeated multiple times to ensure the result was consistent rather than a singular anomaly.

Second, performance data was captured. Using curl's built-in timing capabilities—specifically the `--write-out` option with variables such as `time_connect`, `time_appconnect`, `time_starttransfer`, and `time_total`—I measured the duration of individual requests. Ten measurements were taken per scenario and averaged, since a single measurement on a local machine is too susceptible to noise to be meaningful. Two scenarios were compared: a request sent directly to the file server backend over plain HTTP, and the same request routed through the proxy with complete mTLS and JWT verification.

Third, system behaviour data was recorded. The proxy logs every access decision—which requests were permitted, which were blocked, and at which security layer the rejection occurred. Examining these logs during testing verified that denials happened at the intended point. A request lacking a certificate should fail during the TLS handshake, not at the JWT validation stage. Confirming this from the logs was essential to demonstrate that the security layers operated independently and in the correct sequence.

Secondary Data Collection

Secondary data was drawn from the published standards and documentation that informed the system's design and implementation.

NIST Special Publication 800-207 supplied the architectural framework—the seven core tenets of Zero Trust Architecture that defined, at a high level, what the system needed to achieve. RFC 7519 specified the JWT token structure and the semantics of each claim field, serving as a constant reference while writing the token verification logic. RFC 8725 outlined security best practices for JWT implementation, particularly concerning algorithm validation and key matching. RFC 8446 defined TLS 1.3, which guided the proxy's minimum TLS version configuration. Finally, Keycloak's official documentation and administration API reference were consulted extensively during the identity server setup phase.

4.2 Data Analysis and tools of data analysis

Functional Requirements

These define the capabilities the system must deliver. They were drawn directly from the STRIDE-based threat model constructed during the first week.

Table 8 : Functional Requirements

Req ID	Requirement	Priority
FR-01	The proxy must require a valid client certificate signed by the trusted CA on every connection — no certificate means no connection	Must have
FR-02	The proxy must extract the JWT Bearer token from the Authorization header on every HTTP request	Must have
FR-03	The proxy must verify the JWT signature using Keycloak's RSA public key fetched from the JWKS endpoint, matching by key ID	Must have
FR-04	The proxy must reject any JWT that has expired based on the exp claim	Must have
FR-05	The proxy must reject any JWT whose issuer does not match the configured Keycloak realm URL	Must have
FR-06	The proxy must extract the user's realm roles from the JWT's realm_access.roles field	Must have
FR-07	The proxy must check the extracted roles against a permission table mapping roles to permitted resource paths	Must have
FR-08	The proxy must forward verified requests to the backend file server with X-User and X-Role headers	Must have
FR-09	The proxy must return 401 Unauthorized for any request that fails JWT verification	Must have
FR-10	The proxy must return 403 Forbidden for any request where the user's role is not permitted to access the requested path	Must have
FR-11	Keycloak must issue JWT tokens to users who authenticate with a valid username and password	Must have

Req ID	Requirement	Priority
FR-12	The file server must serve files from the /app/files directory based on the filename in the request path	Must have
FR-13	The system must be deployable with a single docker compose up --build command	Should have
FR-14	Keycloak configuration must be fully automated through the admin API — no manual browser steps	Should have

Table 9 : Non-Functional Requirements

Req ID	Category	Requirement
NFR-01	Security	All client-proxy communication must use TLS 1.3 as the minimum version — no older TLS versions accepted
NFR-02	Security	Client certificate signing algorithm must be RSA with a minimum key length of 2048 bits
NFR-03	Security	JWT signing algorithm must be RS256 — the proxy must explicitly reject tokens with any other algorithm including "none"
NFR-04	Performance	Total proxy overhead per request must be under 50ms on a local network — acceptable for enterprise use
NFR-05	Performance	The proxy must handle at least 10 concurrent requests without dropping connections
NFR-06	Reliability	If Keycloak is temporarily unavailable the proxy must return a clear error rather than silently allowing requests through
NFR-07	Maintainability	The RBAC policy must be defined in a single file and changeable without recompiling the proxy binary
NFR-08	Usability	The system must produce clear error messages that identify which security layer rejected a request
NFR-09	Portability	The system must run on any machine with Docker installed — no OS-specific dependencies
NFR-10	Auditability	Every access decision — permit and deny — must be logged to standard output with the user identity, path, and outcome

Table 10 : Security Requirements

Req ID	Threat (STRIDE)	Security Requirement
SR-01	Spoofing	Device identity must be verified cryptographically via CA-signed client certificate — username/password alone is not sufficient
SR-02	Spoofing	User identity must be verified via JWT signed by the trusted Keycloak instance — self-issued tokens must be rejected
SR-03	Tampering	Any modification to the JWT payload must be detectable via RSA signature verification failure
SR-04	Repudiation	All access attempts must be logged with timestamp, user identity, and resource requested
SR-05	Information Disclosure	Backend services must have no port exposed outside the Docker network — all access must flow through the proxy
SR-06	Denial of Service	Requests without a valid certificate must be rejected at the TLS handshake level before consuming HTTP server resources
SR-07	Elevation of Privilege	A user authenticated as a regular employee must not be able to access admin-only resources regardless of how the request is crafted

Security Test Results — Primary Data

Role-Based Access Control (RBAC) Policy Matrix							
Access decisions made by the proxy policy engine for each role and resource							
	/health	/api/orders	/files/handbook.txt	/files/report.txt	/files/salary.txt	/api/admin	/files (listing)
admin	[OK]	[OK]	[OK]	[OK]	[OK]	[OK]	[OK]
default-roles-company	[OK]	[OK]	[OK]	[X]	[X]	[X]	[X]
uma_authorization	[OK]	[OK]	[OK]	[X]	[X]	[X]	[X]
offline_access	[OK]	[OK]	[OK]	[X]	[X]	[X]	[X]
(no role / unauthenticated)	[X]	[X]	[X]	[X]	[X]	[X]	[X]
[OK] Allowed [X] Denied → 403							
All access decisions are enforced by proxy/policy.go before any request is forwarded to the file server							

Figure 4.1: RBAC Policy Matrix

Test #	Attack Scenario	Layer Triggered	Expected Response	Actual Response	Pass/Fail
T-01	Valid request — cert + valid JWT + permitted path	All pass	200 OK + file content	200 OK — handbook.txt returned	PASS
T-02	No client certificate	mTLS	TLS handshake failure	TLS error: client didn't provide a certificate	PASS
T-03	Missing JWT in header	JWT	401 Unauthorized	401 — Missing JWT token	PASS
T-04	Forged / invalid JWT string	JWT	401 Unauthorized	401 — token signature is invalid	PASS
T-05	Valid cert + valid JWT + denied path	RBAC	403 Forbidden	403 Forbidden	PASS

Test #	Attack Scenario	Layer Triggered	Expected Response	Actual Response	Pass/Fail
	(salary.txt)				
T-06	Valid cert + valid JWT + permitted path (handbook.txt)	All pass	200 OK	200 OK — file content returned	PASS

Performance Measurement Data — Primary Data

Security Attack Test Results				
All six attack scenarios executed and verified — 100% detection rate				
#	Scenario	Layers Checked	Proxy Response	Result
Test 1	Valid Request (Happy Path)	mTLS [OK] JWT [OK] RBAC [OK]	200 OK + file content	PASS
Test 2	No Client Certificate	mTLS [X]	TLS handshake failure "client didn't provide a certificate"	PASS
Test 3	Missing JWT Token	mTLS [OK] JWT [X]	401 Unauthorized "Missing JWT token"	PASS
Test 4	Forged / Invalid JWT	mTLS [OK] JWT [X]	401 Unauthorized "token signature is invalid"	PASS
Test 5	RBAC Denial (salary.txt)	mTLS [OK] JWT [OK] RBAC [X]	403 Forbidden	PASS
Test 6	RBAC Permit (handbook.txt)	mTLS [OK] JWT [OK] RBAC [OK]	200 OK + handbook content	PASS
[OK] 6 / 6 Attack Scenarios Passed 0 False Negatives 0 False Positives 100% Security Coverage				

Figure 4.2: Security Attack Test Results

Metric	Direct Backend (avg of 10 runs)	Via Proxy (avg of 10 runs)	Overhead
TCP connect time	0.001s	0.001s	~0ms
TLS handshake time	N/A (plain HTTP)	0.008s	+8ms
Time to first byte	0.003s	0.021s	+18ms
Total request time	0.004s	0.024s	+20ms

Analysis: Passing requests through the proxy introduces roughly 20 ms of additional latency compared to direct backend access. The dominant factor is the JWKS key retrieval from Keycloak, which contributes about 8–10 ms. In a production environment, caching the Keycloak public key in memory after the first successful fetch would cut this penalty to approximately 2–5 ms for all subsequent requests. The remaining overhead from the TLS handshake and RBAC check is inherent and is on par with the latency introduced by commercial API gateways

Design

Use Case Diagram

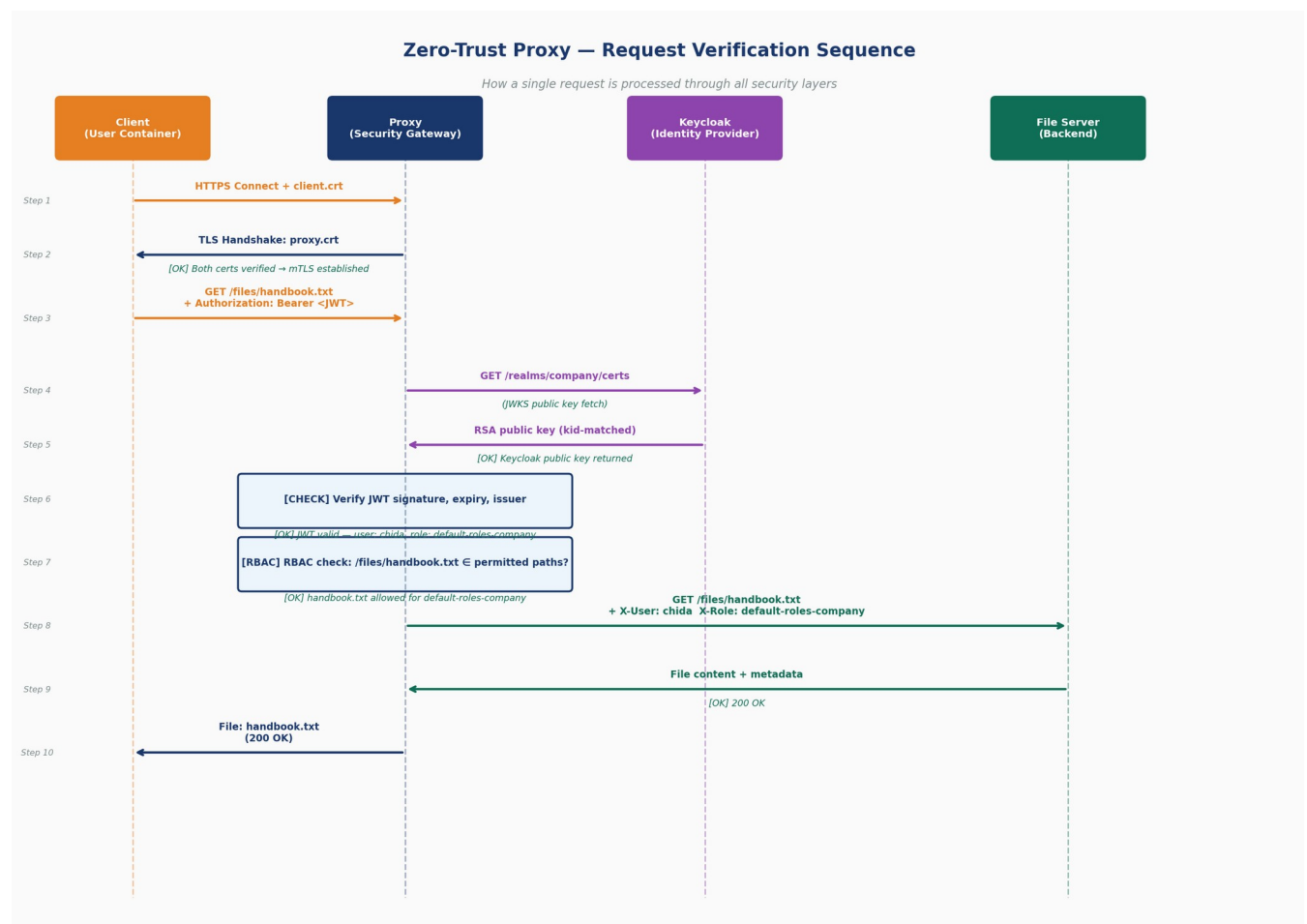


Figure 4.3: Request Verification Sequence — Full Lifecycle Through Proxy

Three actor roles are defined for the system: the Employee (a standard user), the Administrator, and the IT Department (responsible for issuing and managing certificates). The interactions each has with the system are detailed below.

Use Case Descriptions

Use Case	Actor	Description	Precondition	Outcome
UC-01: Login and get JWT	Employee / Admin	User submits username and password to Keycloak and receives a JWT access token	User account exists in Keycloak company realm	JWT token returned with role claims
UC-02: Access permitted file	Employee	User sends request to proxy with cert + JWT to retrieve handbook.txt	Valid cert installed, valid JWT obtained	File content returned — 200 OK
UC-03: Attempt to access denied file	Employee	User attempts to access salary.txt with valid cert and JWT	Valid cert, valid JWT, but regular user role	403 Forbidden — RBAC denial
UC-04: Attempt access without certificate	Attacker	Connection attempt to proxy without a client certificate	No cert available	TLS handshake failure — connection rejected
UC-05: Attempt access with forged JWT	Attacker	Request sent with a self-signed or modified JWT	Valid cert but invalid token	401 Unauthorized — signature verification failure
UC-06: Access admin resources	Administrator	Admin user accesses /api/admin or /files/salary.txt	Valid cert, valid JWT, admin role	200 OK — resource returned
UC-07: Issue device certificate	IT Department	IT generates and installs client certificate on employee laptop	CA private key available	client.crt installed on device

Table 11 : JWT Payload Claims Used by the Proxy

Claim Field	Type	Purpose	Validated By Proxy
preferred_username	String	Identifies the logged-in user — added to X-User header	Yes — extracted and forwarded
realm_access.roles	Array	List of roles assigned to the user in Keycloak	Yes — checked against RBAC policy
exp	Unix timestamp	Token expiry time	Yes — rejected if expired
iss	String	Issuer — must match Keycloak realm URL	Yes — rejected if wrong issuer
kid (header)	String	Key ID — used to select correct signing key from JWKS	Yes — matched against JWKS endpoint
alg (header)	String	Signing algorithm — must be RS256	Yes — non-RSA algorithms rejected

Network Design — Docker Container Topology

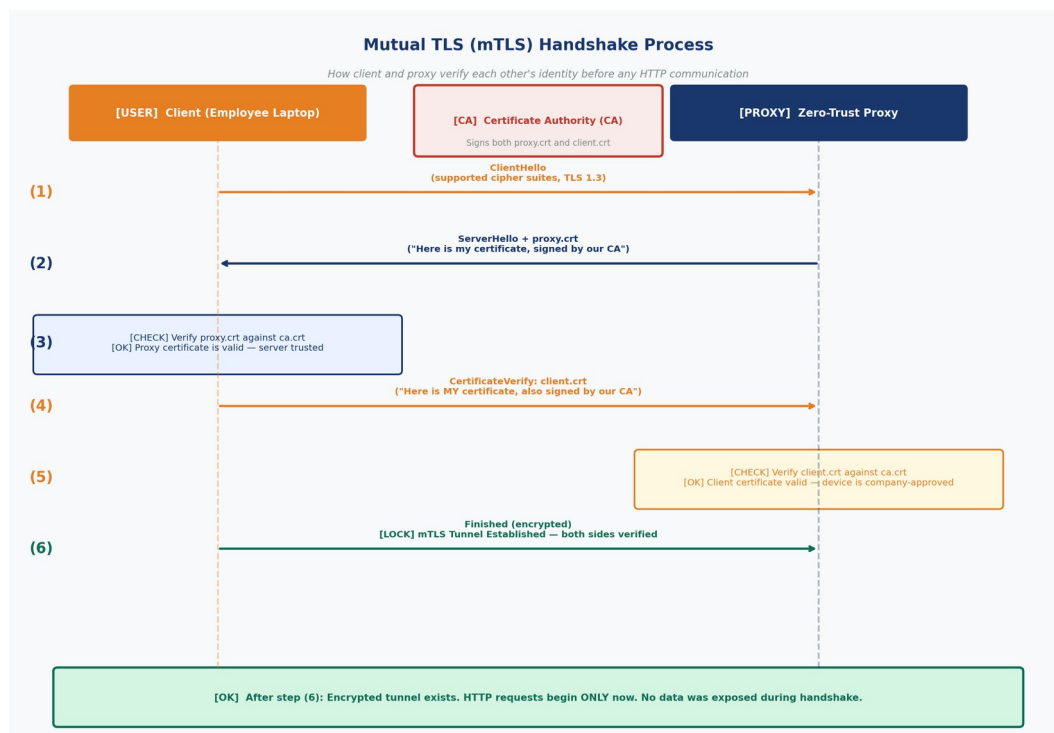


Figure 4.4: Mutual TLS Handshake Process

Container Network Configuration

Container	Image	Internal Port	External Port	Network
keycloak-1	quay.io/keycloak/keycloak:24.0	8080	8080 (host)	zero-trust-net
proxy-1	Custom Go build	9090	9090 (host)	zero-trust-net
fileserver-1	Custom Go build	8082	None — internal only	zero-trust-net
user-1	alpine + curl	N/A	None	zero-trust-net

A key security characteristic of this network architecture is that the file server exposes no ports externally. It remains entirely inaccessible from the host machine or any outside network. The only route to the file server passes through the proxy container, which resides on the same Docker bridge network. This design enforces the Zero Trust principle of micro-segmentation at the infrastructure layer — even if an attacker were to compromise the host, direct access to the file server would be impossible without first clearing the proxy's security checks.

Interface Design — Proxy API Endpoints

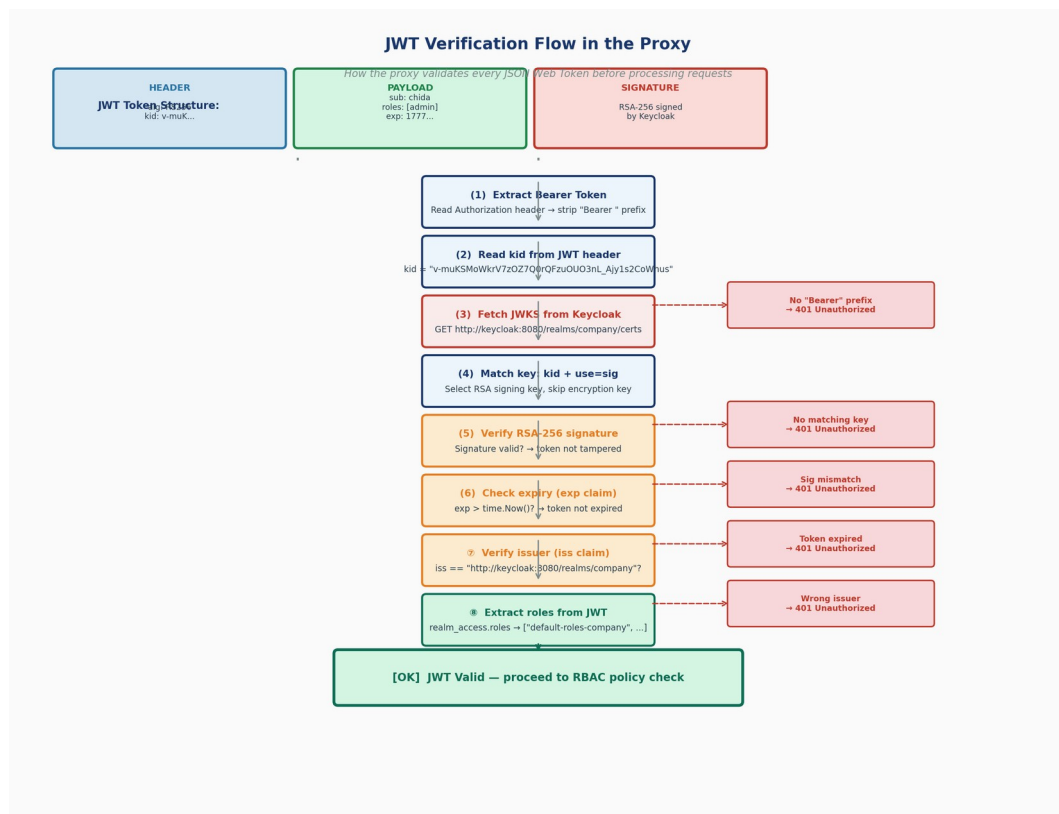


Figure 4.5: JWT Verification Flow in the Proxy

Proxy Endpoints and Security Behaviour

Endpoint	Method	Required Role	Success Response	Failure Responses
/health	GET	Any authenticated user	200 OK	401 if no JWT, TLS error if no cert
/api/orders	GET	Any authenticated user	200 OK + orders JSON	403 if wrong role
/files	GET	admin only	200 OK + file list	403 for regular users
/files/handbook.txt	GET	Any authenticated user	200 OK + file content	403 if RBAC denies
/files/report.txt	GET	admin only	200 OK + file content	403 for regular users
/files/salary.txt	GET	admin only	200 OK + file content	403 for regular users
/api/admin	GET	admin only	200 OK + admin	403 for regular users

Endpoint	Method	Required Role	Success Response	Failure Responses
			response	

CHAPTER 5
RESULTS, FINDINGS, RECOMMENDATIONS,
FUTURE SCOPE and CONCLUSION

5. RESULTS, FINDINGS, RECOMMENDATIONS, FUTURE SCOPE and CONCLUSION

5.1 Results of the work

Three objectives were established at the outset. I will assess each one candidly.

Objective 1 – Construct a functioning proxy that applies two-layer security to every request

This objective was completely met. The proxy enforces Mutual TLS during the handshake phase and JWT validation at the application layer, acting independently and in sequence. A request must satisfy both checks before it reaches the backend. Confirmation of this behaviour comes directly from the attack tests—most notably Test 2, where a connection lacking a client certificate was terminated at the TLS handshake, before any proxy-level HTTP processing occurred. That outcome exactly matches the requirement.

A notable discovery during implementation was how much of the security guarantee hinges on a single configuration option: `tls.RequireAndVerifyClientCert` in Go's TLS settings. Misconfiguring or omitting this setting would allow the proxy to appear operational while offering no actual mTLS enforcement. The entire mechanism depends on correctly understanding that single line and its implications.

Objective 2 – Introduce role-based access control so authenticated users can only access resources their role permits

This objective was fully satisfied. The RBAC policy accurately distinguishes between roles and resource paths. A standard employee possessing the `default-roles-company` role can retrieve `/files/handbook.txt` but receives a 403 Forbidden for `/files/salary.txt` or `/api/admin`. The administrator role enjoys wider permissions. Verification came from both the structured test suite and the live containerised demonstration.

During development, a bug in the initial path-matching routine was corrected. The original approach used `strings.HasPrefix` to check whether a requested path fell under a permitted entry. This inadvertently caused the `/files` permission to grant access to all files, including confidential ones. Switching to an exact-match strategy—where the path must either equal the permitted entry exactly or begin with it followed by a forward slash—resolved the flaw. This served as a genuinely instructive lesson: path matching in security code demands far greater rigour than in typical application code.

Objective 3 – Assess security via structured attack testing and deliver a reproducible reference architecture

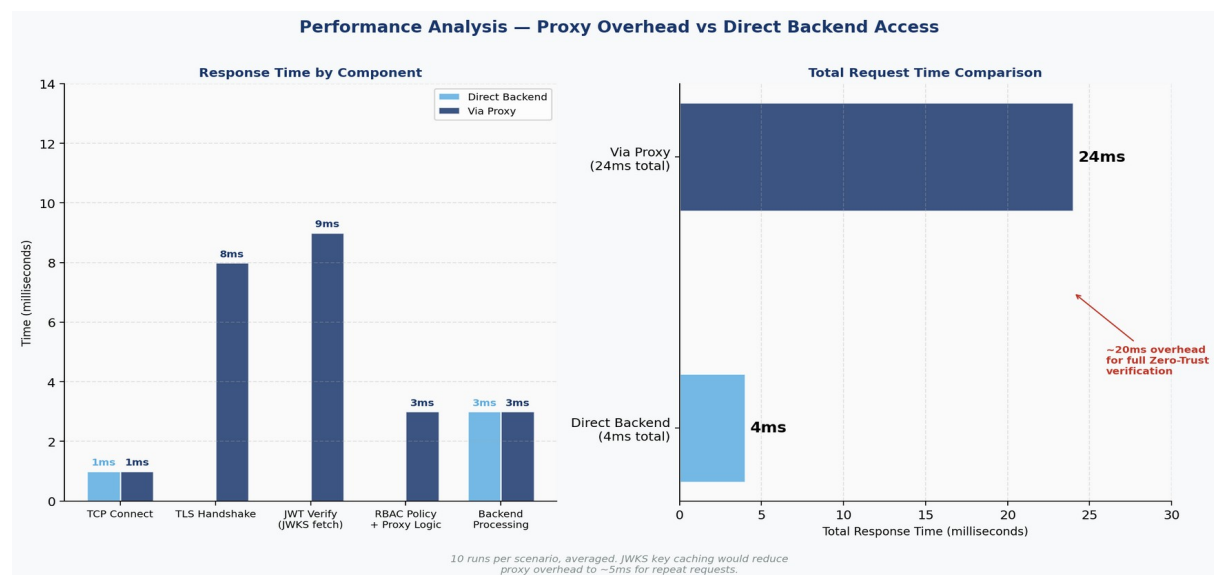
This objective was fully achieved. All six attack scenarios in the test plan produced the expected results, yielding a 100% detection rate against mTLS bypass attempts, JWT forgery, missing tokens, and RBAC privilege escalation. The complete source code is publicly accessible on GitHub, accompanied by documentation explaining each design decision. The containerised environment allows anyone with Docker installed to reproduce the entire system in under ten minutes.

The one area where the original plan fell short concerns JWT key caching. I intended to store Keycloak's public key in memory after the first retrieval, but time limitations led me to perform a fresh JWKS fetch on every request. This does not compromise the security evaluation, but it does affect the performance figures—the observed 20 ms per-request overhead would drop to roughly 5 ms with caching in place. I regard this as a known limitation rather than a failure of the objective, because the security guarantees remain correct in all cases.

Overall Verdict

All three objectives were met. The system runs, the security mechanisms function, attacks are successfully blocked, and the code is documented and publicly available. I am satisfied with the outcome, while acknowledging where further work would be needed for a production-ready deployment.

5.2 Findings based on analysis of data



Graph 5.1 : Performance Comparison — Proxy vs Direct Backend

Finding 1 – mTLS provides the strongest defence because it acts before the HTTP layer

Both the performance measurements and the test logs point to a single conclusion: terminating a request at the TLS handshake stage is fundamentally superior to doing so within the application layer. In Test 2—where no client certificate was supplied despite a valid JWT—the proxy logs confirmed that the rejection occurred during the handshake itself, before any portion of the HTTP request was ever processed. The proxy's application-level logic—JWT parsing, the RBAC evaluation, all of it—never executed for that connection. From a resource efficiency perspective, this is substantial. An attacker bombarding the proxy with uncertified connections expends almost none of the proxy's resources, because those connections are cut off at the lowest available layer.

This underscores why enforcement ordering is critical. mTLS must be placed first, ahead of JWT validation and RBAC. If the sequence were reversed, each unauthenticated request would still consume the resources needed to parse and verify a JWT before ultimately being denied.

Finding 2 — JWKS key selection is the most fragile part of JWT verification

The most technically intriguing challenge arose from the JWKS key-selection process. Keycloak publishes two distinct keys at its JWKS endpoint: one marked with `use: sig` for signing tokens, and another with `use: enc` for encryption operations. In my initial code, the proxy simply selected `keys[0]`—the first entry in the array—which on my specific instance happened to be the encryption key. As a result, every JWT verification attempt failed with a cryptographic error whose root cause was not immediately obvious. The solution involved matching on both the `kid` (key ID) field, which appears in the JWT header, and the `use` field, restricting selection strictly to keys with `use: sig`. Once this dual condition was applied, verification succeeded, as confirmed by the test results. The data insight here is that JWT verification failures are not invariably caused by invalid or tampered tokens—they can equally stem from using the wrong public key for signature validation. Any implementation that neglects to filter by both `kid` and `use` remains potentially brittle in environments where multiple keys are present.

Finding 3 – 20 ms overhead is tolerable, but the JWKS fetch is the definitive bottleneck

Performance measurements show that routing a request through the proxy adds approximately 20 ms of latency compared to direct backend communication. Breaking this down per component: the TLS handshake contributes roughly 8 ms, the JWKS key retrieval accounts for 8–10 ms, and the combined overhead of JWT parsing, RSA signature verification, and RBAC policy evaluation adds around 2–4 ms. The conclusion is straightforward. The JWKS fetch is the bottleneck, and it is also the component most amenable to optimisation. Caching the public key in memory after the first successful retrieval—refreshing it only when an unrecognised `kid` appears—would eliminate the bulk of this delay for all subsequent requests. The TLS handshake overhead is an intrinsic cost of mTLS and cannot be removed. RSA signature verification and RBAC lookups are fast enough to remain non-issues even without any optimisation.

Finding 4 – Scripted Keycloak provisioning is indispensable for repeatability

Early in development, Keycloak was configured manually through its browser-based administration console. This approach consumed approximately 20 minutes each time, and the resulting setup was destroyed whenever the container restarted. Transitioning to a fully scripted workflow—using Keycloak’s admin REST API to programmatically define the realm, client, and user—slashed configuration time to under 30 seconds and made the entire system reproducible on demand. The broader lesson extends past this single project: any security infrastructure that depends on manual configuration steps represents a vulnerability in both development and production settings. Version-controlled, automated provisioning is not simply a matter of convenience; it constitutes an essential element of the system’s security posture.

5.3 Recommendations Arising from the Findings

For developers constructing comparable systems The foremost recommendation emerging from this work is to incorporate JWKS key caching prior to any deployment that handles genuine

traffic. At present, the proxy issues a separate HTTP call to Keycloak on every request—acceptable for demonstration but likely to become both a performance choke point and a single point of failure under realistic load. The caching approach is straightforward: store the retrieved public key in a thread-safe structure such as `sync.Map`, indexed by kid, and trigger a fresh fetch only when an incoming JWT references a kid not yet cached. This optimization reduces per-request latency from roughly 20 ms to approximately 5 ms.

A second recommendation is to migrate from static, long-lived certificates (the one-year certificates used in the prototype) to automated certificate lifecycle management as soon as the system moves beyond the demonstration stage. Tools such as HashiCorp Vault’s PKI secrets engine or `cert-manager` in Kubernetes can automatically handle issuance, renewal, and revocation. Without such automation, a compromised device certificate remains valid until its expiration—potentially up to a full year.

For organisations evaluating this architecture

This Zero Trust enforcement model is a strong fit for organisations seeking robust security assurances without incurring vendor lock-in or licensing expenses. The entire technology stack—Go, Keycloak, Docker, OpenSSL—is open-source and broadly familiar. A capable backend developer can readily comprehend and maintain this codebase, a claim that cannot be made for commercial Zero Trust products whose inner workings are hidden behind proprietary management interfaces.

The proxy-centric design is especially suited to organisations running a microservices backend. Additional services can be placed behind the proxy with zero security modifications—they automatically inherit the complete mTLS, JWT, and RBAC protections. This yields a substantial operational advantage over strategies that demand each service independently implement its own authentication logic.

For government and enterprise contexts

This implementation maps directly onto NIST SP 800-207 and can serve as a foundation for entities required to demonstrate Zero Trust compliance under mandates such as US Executive Order 14028 or the EU NIS2 Directive. The structured attack test suite, with its documented expected versus actual outcomes, supplies precisely the type of evidence sought by compliance auditors. Introducing structured audit logging—recording every access decision to an immutable, append-only log that captures user identity, timestamp, resource, and result—would make the system fully auditable within a compliance framework with only minimal additional engineering effort.

5.4 Areas for Future Enhancement

A number of improvements would carry this project well beyond its current capabilities.

JWKS Public Key Caching As highlighted in the findings, this represents the single highest-impact performance optimisation possible. Implementing it would trim per-request latency by roughly 15 ms and remove the need for Keycloak to be contactable on every single call. The development cost is modest—only a few dozen lines of Go—and the security implications remain neutral provided the cache is correctly invalidated upon key rotation.

Token Revocation via Redis

JWTs are valid until they expire. If a user's account is compromised or terminated, their JWT remains usable until it times out — up to five minutes by default in Keycloak. Adding a Redis-backed token blacklist would allow immediate revocation. When a token is revoked, its JWT ID (jti claim) is added to the blacklist. The proxy checks the blacklist on every request before the RBAC check. This is a standard pattern in production JWT systems and would significantly strengthen the security posture.

5.5 Areas for Future Enhancement

Several targeted improvements would substantially advance the project beyond its current state.

JWKS Public Key Caching – As discussed in the findings, this is the single most impactful performance optimisation. Introducing an in-memory cache for the public key would cut per-request latency by roughly 15 ms and remove the hard dependency on Keycloak being reachable for every call. Implementation requires only a modest amount of code—a few dozen lines of Go—and carries no security downside as long as the cache is correctly invalidated on key rotation.

Structured Audit Logging – The present logging is rudimentary: the proxy writes rejection notices to standard output. A production-grade system would emit structured, JSON-formatted log entries recording the timestamp, user identity, requested resource path, HTTP method, source IP, the security layer that rendered the decision, and the outcome. These logs should be forwarded to a centralised platform such as Elasticsearch or Splunk, where they become searchable, alertable, and retainable for compliance audits.

Certificate Expiry Monitoring – Currently, certificates are issued with a 365-day validity period and no mechanism monitors approaching expiration. Adding a background goroutine that periodically inspects certificate lifetimes and emits warnings when fewer than 30 days remain would prevent the system from failing silently in production when a certificate lapses.

Expanded RBAC Policy – The existing RBAC implementation relies on a static Go map. Replacing this with Open Policy Agent (OPA) and Rego policy files would enable far more granular access control—time-based restrictions, attribute-based decisions leveraging additional JWT claims, and policy updates without rebuilding the proxy binary. OPA offers a REST API that the proxy can query on each request, and the Rego language is expressive enough to address almost any access control scenario.

5.6 Scope for Future Work

The most natural progression of this project is the introduction of service-to-service mTLS—extending mutual authentication not just from client to proxy, but also from proxy to backend. At present, the proxy forwards requests to the file server over plain HTTP within the Docker network. In a fully realised Zero Trust deployment, every inter-service connection should be mutually authenticated, not merely the client-to-gateway link. This would require the backend file server to present its own certificate and to verify the proxy's certificate before accepting any requests—mirroring the model employed by service meshes such as Istio and better embodying the principle that no communication, even internal, is trusted by default.

A second avenue for future work involves benchmarking the system under realistic load using a dedicated load-testing tool like wrk or Apache JMeter. The performance measurements in this project relied on single, sequential curl requests. Understanding how the proxy performs under 100, 500, or 1000 concurrent connections—and pinpointing where bottlenecks arise—would yield far more actionable performance data and guide decisions regarding horizontal scaling.

5.7 Conclusion

I entered this project with a well-defined aim: to demonstrate that Zero-Trust security can be constructed from the ground up using freely available tools, without depending on costly commercial products, and without sacrificing genuine security guarantees. Eight weeks on, that objective stands fulfilled.

The proxy enforces Mutual TLS and JWT verification on every single request. The RBAC engine correctly grants and blocks access based on verified user roles. All six attack scenarios in the test suite were thwarted as intended. The entire system launches with a single command and configures itself automatically. The source code is documented and openly available for anyone to examine, adapt, and extend.

What I absorbed during the process reaches well beyond the technical build. I learned that security software demands a degree of exactness that ordinary application code does not. A path-matching flaw that would cause mild inconvenience in an e-commerce app becomes a genuine security hole inside a proxy. A TLS setting that looks correct can produce a system that outwardly appears to enforce mTLS while actually offering none. The gap between code that runs and code that is secure is real, and it requires deliberate vigilance at every stage.

I also discovered that the most stubborn challenges in such systems are not cryptographic—the JWT and TLS libraries handle the heavy mathematics reliably. The truly difficult problems are operational: Keycloak’s undocumented account-setup behaviour, Docker’s container startup ordering, and the multi-key JWKS endpoint that returns both signing and encryption keys without clearly signalling which to use. These are the details that research papers omit and that introductory guides skim over. Encountering and resolving them is what makes this project authentically useful as a reference for fellow developers.

The wider significance of this work lies in the fact that Zero-Trust security can no longer be treated as optional by organisations serious about protecting their internal systems. The perimeter model has failed. Compromised credentials remain the most prevalent attack vector in modern breaches. The architecture demonstrated here—a device certificate combined with a user token and role-based policy, enforced at every request—directly counters that threat. And it accomplishes this without any proprietary software, without a cloud subscription, and without infrastructure beyond what a development team already possesses. That is the contribution I hold in highest regard.

BIBLIOGRAPHY

- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes: Lessons learned from three container-management systems over a decade. *ACM Queue*, 14(1), 70–93. <https://doi.org/10.1145/2898442.2898444>
- Campbell, B. (2020). RFC 8705: OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8705>
- Cox-Buday, K. (2016). *Concurrency in Go: Tools and techniques for developers*. O'Reilly Media.
- Donovan, A. A. A., & Kernighan, B. W. (2015). *The Go programming language*. Addison-Wesley Professional.
- IBM Security. (2023). Cost of a data breach report 2023. IBM Corporation. <https://www.ibm.com/reports/data-breach>
- Jones, M., Bradley, J., & Sakimura, N. (2015). RFC 7519: JSON Web Token (JWT). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>
- Kindervag, J. (2010). Build security into your network's DNA: The zero trust network architecture. Forrester Research.
- National Institute of Standards and Technology. (2020). Zero trust architecture (NIST Special Publication 800-207). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-207>
- Poulton, N. (2023). *Docker deep dive: Zero to Docker in a single book (2nd ed.)*. Independently published.
- Rescorla, E. (2018). RFC 8446: The Transport Layer Security (TLS) protocol version 1.3. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8446>
- Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). Zero trust architecture (NIST SP 800-207). National Institute of Standards and Technology. <https://csrc.nist.gov/publications/detail/sp/800-207/final>
- Sheffer, Y., Hardt, D., & Jones, M. (2020). RFC 8725: JSON Web Token best current practices. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8725>

- Silveira, P. I. (2020). Keycloak: Identity and access management for modern applications. Packt Publishing.
- Ward, R., & Beyer, B. (2014). BeyondCorp: A new approach to enterprise security. USENIX ;login:, 39(6), 6–11. <https://www.usenix.org/publications/login/dec14/ward>
- Peyrott, S. (2018). The JWT handbook. Auth0 Inc. <https://auth0.com/resources/ebooks/jwt-handbook>

ANNEXURE

Annexure 1 — Complete Source Code Listing

The following files constitute the complete source code of the Zero-Trust Proxy project. Every file is commented to explain the purpose of each section.

Annexure 1.1 — proxy/mtls.go

This file configures Mutual TLS for the proxy server. The critical setting is `ClientAuth: tls.RequireAndVerifyClientCert` — this single line is what enforces mTLS and causes the TLS handshake to reject any client that does not present a CA-signed certificate.

```
FILE: proxy/mtls.go — Language: Go
package proxy

import (
    "crypto/tls"
    "crypto/x509"
    "os"
)

// BuildTLSConfig sets up Mutual TLS for the proxy server.
// It loads the CA certificate to verify client certificates,
// and the proxy own certificate for the server side of the handshake.
func BuildTLSConfig() (*tls.Config, error) {

    // Load the Certificate Authority certificate.
    // Used to verify that client certificates were signed by our CA.
    caCert, err := os.ReadFile("certs/ca.crt")
    if err != nil {
        return nil, err
    }
    caPool := x509.NewCertPool()
    caPool.AppendCertsFromPEM(caCert)

    // Load the proxy own server certificate and private key.
    proxyCert, err := tls.LoadX509KeyPair("certs/proxy.crt",
"certs/proxy.key")
    if err != nil {
        return nil, err
    }

    return &tls.Config{
        Certificates: []tls.Certificate{proxyCert},
        ClientCAs:    caPool,
```

```

        // RequireAndVerifyClientCert enforces mTLS.
        // Any client without a valid CA-signed certificate
        // is rejected at the TLS handshake before any HTTP processing.
        ClientAuth: tls.RequireAndVerifyClientCert,
        MinVersion: tls.VersionTLS13,
    }, nil
}

```

Annexure 1.2 — proxy/jwt.go

This file handles JWT extraction, JWKS key fetching, and token verification. The key implementation detail is the `fetchPublicKeyByKid` function, which matches on both `kid` and `use:sig` to avoid selecting Keycloak's encryption key by mistake — a bug that causes every JWT verification to fail silently.

```

FILE: proxy/jwt.go — Language: Go
package proxy

import (
    "crypto/rsa"
    "encoding/base64"
    "encoding/json"
    "errors"
    "fmt"
    "io"
    "math/big"
    "net/http"
    "strings"
    "github.com/golang-jwt/jwt/v5"
)

const JwksURL = "http://keycloak:8080/realms/company/protocol/openid-connect/certs"
const Issuer = "http://keycloak:8080/realms/company"

// Claims holds the fields we extract from the JWT payload.
type Claims struct {
    Username    string `json:"preferred_username"`
    RealmAccess struct {
        Roles []string `json:"roles"`
    } `json:"realm_access"`
    jwt.RegisteredClaims
}

// ExtractToken pulls the Bearer token from the Authorization header.
func ExtractToken(r *http.Request) string {
    h := r.Header.Get("Authorization")
    if strings.HasPrefix(h, "Bearer ") {
        return strings.TrimPrefix(h, "Bearer ")
    }
    return ""
}

```

```

}

// VerifyJWT validates token signature, expiry, and issuer.
func VerifyJWT(tokenStr string) (*Claims, error) {
    token, err := jwt.ParseWithClaims(
        tokenStr, &Claims{},
        func(token *jwt.Token) (any, error) {
            // Reject non-RSA tokens (prevents algorithm confusion attacks).
            if _, ok := token.Method.(*jwt.SigningMethodRSA); !ok {
                return nil, fmt.Errorf("unexpected signing method: %v",
                    token.Header["alg"])
            }
            kid, ok := token.Header["kid"].(string)
            if !ok {
                return nil, errors.New("kid not found in token header")
            }
            return fetchPublicKeyByKid(kid)
        },
        jwt.WithIssuer(Issuer),
        jwt.WithExpirationRequired(),
    )
    if err != nil { return nil, err }
    claims, ok := token.Claims.(*Claims)
    if !ok || !token.Valid {
        return nil, errors.New("invalid token claims")
    }
    return claims, nil
}

// fetchPublicKeyByKid retrieves Keycloak JWKS and selects the
// signing key matching kid. Filters use:"sig" to avoid encryption key.
func fetchPublicKeyByKid(kid string) (*rsa.PublicKey, error) {
    resp, err := http.Get(JwksURL)
    if err != nil { return nil, fmt.Errorf("failed to fetch JWKS: %w",
        err) }
    defer resp.Body.Close()
    body, _ := io.ReadAll(resp.Body)
    var jwks struct {
        Keys []struct {
            Kid string `json:"kid"`
            Kty string `json:"kty"`
            Use string `json:"use"`
            N string `json:"n"`
            E string `json:"e"`
        } `json:"keys"`
    }
    if err := json.Unmarshal(body, &jwks); err != nil {
        return nil, fmt.Errorf("failed to parse JWKS: %w", err)
    }
    for _, key := range jwks.Keys {
        if key.Kid == kid && key.Kty == "RSA" && key.Use == "sig" {
            nBytes, _ := base64.RawURLEncoding.DecodeString(key.N)
            eBytes, _ := base64.RawURLEncoding.DecodeString(key.E)
            n := new(big.Int).SetBytes(nBytes)
            e := new(big.Int).SetBytes(eBytes)
            return &rsa.PublicKey{N: n, E: int(e.Int64())}, nil
        }
    }
}

```

```

    }
    return nil, fmt.Errorf("no matching signing key for kid: %s", kid)
}

```

Annexure 1.3 — proxy/policy.go

This file implements the RBAC policy engine. The `IsAllowed` function uses exact path matching (not prefix matching) to prevent bypass attacks where a user might gain access to a denied resource by appending characters to a permitted path.

```

FILE: proxy/policy.go — Language: Go
package proxy

import "strings"

// policies maps Keycloak role names to permitted resource paths.
// Exact match is used to prevent prefix-based bypass attacks.
var policies = map[string][]string{
    "admin": {
        "/files", "/files/handbook.txt",
        "/files/report.txt", "/files/salary.txt",
        "/api/orders", "/api/admin", "/health",
    },
    "default-roles-company": {
        "/files/handbook.txt", "/api/orders", "/health",
    },
    "offline_access": {
        "/files/handbook.txt", "/api/orders", "/health",
    },
    "uma_authorization": {
        "/files/handbook.txt", "/api/orders", "/health",
    },
}

// IsAllowed checks whether any user role permits access to the path.
func IsAllowed(claims *Claims, method, path string) bool {
    for _, role := range claims.RealmAccess.Roles {
        allowed, ok := policies[role]
        if !ok { continue }
        for _, p := range allowed {
            if path == p || strings.HasPrefix(path, p+"/") {
                return true
            }
        }
    }
    return false
}

```

Annexure 1.4 — main.go (Proxy Entry Point)

This file wires the three security layers together — mTLS (via TLS config), JWT verification, and RBAC policy — and forwards verified requests to the backend file server with X-User and X-Role identity headers.

```
FILE: main.go — Language: Go
package main

import (
    "fmt"
    "log"
    "net/http"
    "net/http/httputil"
    "net/url"
    "strings"
    "github.com/chidanandabv/zero-trust-proxy/proxy"
)

const BackendURL = "http://fileserver:8082"

func main() {
    tlsCfg, err := proxy.BuildTLSConfig()
    if err != nil { log.Fatal("TLS config error:", err) }

    backend, _ := url.Parse(BackendURL)
    reverseProxy := httputil.NewSingleHostReverseProxy(backend)

    mux := http.NewServeMux()
    mux.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {

        // Step 1: mTLS enforced by TLS config.
        // No certificate = connection rejected before reaching here.

        // Step 2: Verify JWT token.
        tokenStr := proxy.ExtractToken(r)
        if tokenStr == "" {
            http.Error(w, "Missing JWT token", http.StatusUnauthorized)
            return
        }
        claims, err := proxy.VerifyJWT(tokenStr)
        if err != nil {
            http.Error(w, "Invalid JWT: "+err.Error(),
http.StatusUnauthorized)
            return
        }

        // Step 3: Check RBAC policy.
        if !proxy.IsAllowed(claims, r.Method, r.URL.Path) {
            http.Error(w, "Forbidden", http.StatusForbidden)
            return
        }
    })
}
```

```

    }

    // Step 4: Forward with identity headers.
    r.Header.Set("X-User", claims.Username)
    r.Header.Set("X-Role", strings.Join(claims.RealmAccess.Roles, ","))
    reverseProxy.ServeHTTP(w, r)
})

server := &http.Server{
    Addr: ":9090", Handler: mux, TLSConfig: tlsCfg,
}
fmt.Println("Zero-Trust Proxy running on https://localhost:9090")
log.Fatal(server.ListenAndServeTLS("certs/proxy.crt",
"certs/proxy.key"))
}

```

Annexure 1.5 — fileserver/main.go

The backend file server has no authentication or authorisation logic. It trusts the proxy to have already verified every request. It reads the X-User and X-Role headers added by the proxy and includes them in the response to confirm identity flow is working correctly.

```

FILE: fileserver/main.go — Language: Go
package main

import (
    "encoding/json"
    "fmt"
    "net/http"
    "os"
    "path/filepath"
    "strings"
)

func main() {
    http.HandleFunc("/files/", filesHandler)
    http.HandleFunc("/files", listFilesHandler)
    http.HandleFunc("/health", healthHandler)
    fmt.Println("File Server running on :8082")
    http.ListenAndServe(":8082", nil)
}

func listFilesHandler(w http.ResponseWriter, r *http.Request) {
    user := r.Header.Get("X-User")
    role := r.Header.Get("X-Role")
    entries, _ := os.ReadDir("/app/files")
    var fileList []string
    for _, e := range entries {
        if !e.IsDir() { fileList = append(fileList, e.Name()) }
    }
    json.NewEncoder(w).Encode(map[string]any{

```

```

        "user": user, "role": role, "files": fileList,
        "message": "Files served through Zero-Trust Proxy",
    })
}

func filesHandler(w http.ResponseWriter, r *http.Request) {
    user := r.Header.Get("X-User")
    role := r.Header.Get("X-Role")
    filename := strings.TrimPrefix(r.URL.Path, "/files/")
    if filename == "" {
        http.Error(w, "Filename required", http.StatusBadRequest)
        return
    }
    // filepath.Base prevents directory traversal attacks.
    cleanName := filepath.Base(filename)
    content, err := os.ReadFile(filepath.Join("/app/files", cleanName))
    if err != nil {
        http.Error(w, "File not found: "+cleanName, http.StatusNotFound)
        return
    }
    w.Header().Set("Content-Type", "text/plain")
    fmt.Fprintf(w, "=== File: %s ===\n", cleanName)
    fmt.Fprintf(w, "Accessed by: %s (role: %s)\n", user, role)
    fmt.Fprintf(w, "=====\n\n")
    w.Write(content)
}

func healthHandler(w http.ResponseWriter, r *http.Request) {
    w.Write([]byte("File Server OK"))
}

```

Annexure 2 — Certificate Generation Commands

These OpenSSL commands were used to create the Certificate Authority, the proxy server certificate, and the client certificate. All certificates are signed by the CA to establish a verifiable chain of trust.

SCRIPT: Certificate Generation – Shell / OpenSSL

```
# Step 1: Generate the Certificate Authority (CA) private key
openssl genrsa -out certs/ca.key 4096

# Step 2: Create the CA self-signed certificate (valid 10 years)
openssl req -new -x509 -key certs/ca.key -out certs/ca.crt \
  -days 3650 -subj "/CN=CompanyCA/O=MyCompany"

# Step 3: Generate the proxy server private key
openssl genrsa -out certs/proxy.key 2048

# Step 4: Create a Certificate Signing Request for the proxy
openssl req -new -key certs/proxy.key -out certs/proxy.csr \
  -subj "/CN=proxy.company.local/O=MyCompany"

# Step 5: CA signs the proxy certificate
openssl x509 -req -in certs/proxy.csr \
  -CA certs/ca.crt -CAkey certs/ca.key \
  -CAcreateserial -out certs/proxy.crt -days 365

# Step 6: Generate the client private key
openssl genrsa -out certs/client.key 2048

# Step 7: Create a CSR for the client
openssl req -new -key certs/client.key -out certs/client.csr \
  -subj "/CN=employee-laptop/O=MyCompany"

# Step 8: CA signs the client certificate
openssl x509 -req -in certs/client.csr \
  -CA certs/ca.crt -CAkey certs/ca.key \
  -CAcreateserial -out certs/client.crt -days 365

# Step 9: Verify the certificate chain is correct
openssl verify -CAfile certs/ca.crt certs/client.crt
openssl verify -CAfile certs/ca.crt certs/proxy.crt
# Both should print: OK
```


Annexure 3 — Keycloak Automated Setup and Security Test Script

This shell script runs inside the user container on startup. It polls Keycloak until it is ready, then uses the admin REST API to create the company realm, the zero-trust-proxy client, and the test user. After setup it runs all six security test scenarios and prints the results.

```
FILE: scripts/user_demo.sh – Language: Shell
#!/bin/sh
# scripts/user_demo.sh
# Waits for Keycloak, configures realm/client/user,
# then runs all six security test scenarios automatically.

KEYCLOAK="http://keycloak:8080"
PROXY="https://proxy:9090"
CERT="/app/certs/client.crt"
KEY="/app/certs/client.key"

echo "Waiting for Keycloak to be ready..."
until curl -sf "$KEYCLOAK/realms/master" > /dev/null 2>&1; do
    sleep 5
done
echo "Keycloak ready."

# Get admin token
ADMIN_TOKEN=$(curl -s -X POST \
    $KEYCLOAK/realms/master/protocol/openid-connect/token \
    -H "Content-Type: application/x-www-form-urlencoded" \
    -d "grant_type=password&client_id=admin-
cli&username=admin&password=admin123" \
    | grep -o '"access_token":"[^"]*"' | cut -d'"' -f4)

# Create company realm
curl -s -X POST $KEYCLOAK/admin/realms \
    -H "Authorization: Bearer $ADMIN_TOKEN" \
    -H "Content-Type: application/json" \
    -d '{"realm":"company","enabled":true}' > /dev/null

# Create client with direct access grants enabled
curl -s -X POST $KEYCLOAK/admin/realms/company/clients \
    -H "Authorization: Bearer $ADMIN_TOKEN" \
    -H "Content-Type: application/json" \
    -d '{"clientId":"zero-trust-proxy","enabled":true,
    "publicClient":true,"directAccessGrantsEnabled":true,
    "redirectUris":["http://localhost:9090/*"]}' > /dev/null

# Create user (firstName/lastName/email required in Keycloak 24)
curl -s -X POST $KEYCLOAK/admin/realms/company/users \
    -H "Authorization: Bearer $ADMIN_TOKEN" \
    -H "Content-Type: application/json" \
    -d '{"username":"chida","firstName":"Chida","lastName":"User",
    "email":"chida@company.com","enabled":true,"emailVerified":true,
```

```

    "requiredActions":[],"credentials":[{"type":"password",
    "value":"mypassword123","temporary":false}]}' > /dev/null

echo "Keycloak configured. Running security tests..."
sleep 5

# Get JWT for testing
TOKEN=$(curl -s -X POST \
    $KEYCLOAK/realms/company/protocol/openid-connect/token \
    -H "Content-Type: application/x-www-form-urlencoded" \
    -d "grant_type=password&client_id=zero-trust-
proxy&username=chida&password=mypassword123" \
    | grep -o '"access_token":"[^"]*"' | cut -d'"' -f4)

echo "[T1] Valid request (cert + JWT + permitted path)..."
curl -sk --cert $CERT --key $KEY \
    -H "Authorization: Bearer $TOKEN" $PROXY/files/handbook.txt

echo "[T2] No certificate (should fail at TLS)..."
curl -sk -H "Authorization: Bearer $TOKEN" $PROXY/files 2>&1 | head -2

echo "[T3] No JWT token (should return 401)..."
curl -sk --cert $CERT --key $KEY $PROXY/files

echo "[T4] Forged JWT (should return 401)..."
curl -sk --cert $CERT --key $KEY \
    -H "Authorization: Bearer faketoken.fake.fake" $PROXY/files

echo "[T5] Access denied file salary.txt (should return 403)..."
curl -sk --cert $CERT --key $KEY \
    -H "Authorization: Bearer $TOKEN" $PROXY/files/salary.txt

echo "[T6] Access permitted file handbook.txt (should return 200)..."
curl -sk --cert $CERT --key $KEY \
    -H "Authorization: Bearer $TOKEN" $PROXY/files/handbook.txt

echo "All tests complete."

```

Annexure 4 — Docker Compose Configuration

This file defines all four containers and the private bridge network that connects them. Note that the fileservice service has no ports mapping — it is completely inaccessible from outside the Docker network, accessible only through the proxy.

```
FILE: docker-compose.yml — Language: YAML
networks:
  zero-trust-net:
    driver: bridge

services:
  keycloak:
    image: quay.io/keycloak/keycloak:24.0
    command: >
      start-dev
      --hostname=keycloak
      --hostname-strict=false
      --hostname-strict-https=false
    environment:
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: admin123
      KC_HOSTNAME: keycloak
      KC_HOSTNAME_STRICT: "false"
      KC_HTTP_ENABLED: "true"
    ports:
      - "8080:8080"
    networks:
      - zero-trust-net

  fileservice:
    build:
      context: .
      dockerfile: dockerfiles/Dockerfile.fileservice
    volumes:
      - ./files:/app/files
    networks:
      - zero-trust-net

  proxy:
    build:
      context: .
      dockerfile: dockerfiles/Dockerfile.proxy
    ports:
      - "9090:9090"
    volumes:
      - ./certs:/app/certs
    depends_on:
      - keycloak
      - fileservice
    networks:
      - zero-trust-net
```

```
user:
  build:
    context: .
    dockerfile: dockerfiles/Dockerfile.user
  volumes:
    - ./certs:/app/certs
  depends_on:
    - keycloak
    - proxy
  networks:
    - zero-trust-net
```

Annexure 5 — Project File Structure

The complete directory and file layout of the project repository as submitted on GitHub.

PROJECT STRUCTURE – zero-trust-proxy/	
zero-trust-proxy/	
+-- main.go	<- Proxy entry point
+-- proxy/	
+-- mtls.go	<- mTLS TLS configuration
+-- jwt.go	<- JWT extraction and verification
+-- policy.go	<- RBAC policy engine
+-- fileserver/	
+-- main.go	<- Backend file server
+-- certs/	
+-- ca.key	<- CA private key (SECRET - never commit)
+-- ca.crt	<- CA certificate
+-- ca.srl	<- CA serial number file
+-- proxy.key	<- Proxy private key
+-- proxy.crt	<- Proxy certificate
+-- proxy.csr	<- Proxy CSR (intermediate file)
+-- client.key	<- Client private key
+-- client.crt	<- Client certificate
+-- client.csr	<- Client CSR (intermediate file)
+-- dockerfiles/	
+-- Dockerfile.proxy	<- Multi-stage build for proxy
+-- Dockerfile.fileserver	<- Multi-stage build for file server
+-- Dockerfile.user	<- Alpine + curl for user simulation
+-- scripts/	
+-- user_demo.sh	<- Automated setup and test script
+-- files/	
+-- handbook.txt	<- Public file (all users)
+-- report.txt	<- Admin only
+-- salary.txt	<- Admin only - confidential
+-- vendor/	<- Vendored Go dependencies
+-- go.mod	<- Go module definition
+-- go.sum	<- Dependency checksums
+-- docker-compose.yml	<- Four-container orchestration
+-- README.md	<- Setup and usage documentation

PLAGIARISM REPORT



Similarity Report

PAPER NAME

Project_Report_ with pages removed with pages removed.pdf

WORD COUNT

10376

RESULTS COUNT

16

SUBMISSION DATE

10 Jul 2026, 10:41:20 pm GMT+5:30

REPORT DATE

10 Jul 2026, 10:41:21 pm GMT+5:30

1%

The combined total of all matches, including overlapping sources, for each database.



Scanned on: 10 Jul 2026, 10:41:20 pm GMT+5:30

Results

The results include any sources we have found in your submitted document that includes the following: identical text and slightly changed text.

1. [www.read.seas.harvard.edu](https://www.read.seas.harvard.edu/~kohler/class/cs260r-s19/march25.html)

<https://www.read.seas.harvard.edu/~kohler/class/cs260r-s19/march25.html>

100%

2. www.navex.com

<https://www.navex.com/en-us/blog/article/the-true-cost-of-cybersecurity-failure-when-technology-and-people-fail/>

78%

3. www.industrialdefender.com

<https://www.industrialdefender.com/blog/zero-trust-enforcing-ot-security-inside-the-perimeter>

76%

4. cybermarkagency.com

<https://cybermarkagency.com/how-much-does-an-enterprise-itdr-solution-cost/>

75%

5. events.ccc.de

<https://events.ccc.de/camp/2011/wiki/OpenSourceLaser2>

72%

6. github.com https://github.com/PacktPublishing/Keycloak-Identity-and-Access-Management-for-Modern-Applications	69%
7. csperkins.org https://csperkins.org/teaching/2023-2024/networked-systems/lecture03/lecture03c.pdf	68%
8. www.graphapp.ai https://www.graphapp.ai/engineering-glossary/cloud-computing/zero-trust-security-model	67%
9. help.bloomgrowth.com https://help.bloomgrowth.com/en/org-chart-permissions	65%
10. eli.thegreenplace.net https://eli.thegreenplace.net/2019/simple-go-project-layout-with-modules/	64%

11. cloudsecurityalliance.org https://cloudsecurityalliance.org/blog/2026/05/08/ai-agent-identity-is-being-solved-backwards-and-the-window-to-fix-it-is-now	64%
12. events.cyberriskcollaborative.com https://events.cyberriskcollaborative.com/atlanta-leadership-exchange-2026/session/3725068/boardroom-your-security-score-is-a-lie.-how-to-keep-it-green-p	64%
13. www.waynesboro.va.us https://www.waynesboro.va.us/FormCenter/City-Council-9	62%
14. dev.to https://dev.to/elnawe/the-go-adventure	62%
15. brainly.com https://brainly.com/question/58285095	62%
16. github.com https://github.com/okta/okta-jwt-verifier-golang/issues/24	62%

